

Mesos - Final Report

Group members: Diana Francesconi, Luca Grecchi, Vadym Kitsul, Andrea Markvukaj

Implemented Features

Mesos is a Java application developed according to the MVC pattern and based on the assigned board game. The project implements the complete game rules, including players, tribes, cards, turns, game phases, final scoring and client-server communication. The application supports both a text-based interface and a JavaFX graphical interface, and allows clients to connect through two different communication protocols: Socket and RMI.

The project structure is divided into packages that reflect the required architecture. The **model** package contains the domain logic, including cards, board, players, tribes, phases and game rules. The **controller** package coordinates requests coming from clients and delegates actions to the current game phase. The **view** package contains both TUI and GUI implementations, while the **network** package abstracts remote communication. Graphical assets, FXML files and JSON configuration files for cards and board elements are located in **src/main/resources**.

In addition to the main requirements, two advanced features were implemented: a database-backed match leaderboard and game state persistence. The leaderboard stores completed match results in a PostgreSQL database. For each player, the system stores the nickname, final score, number of players in the match and match date. At the end of the game, the client receives the player's position in the leaderboard filtered by player count and can display the complete ranking for that match format.

The persistence feature allows the server to periodically save the current game state to disk. In case of a server crash, when the server restarts it attempts to load the saved game and enters recovery mode. Players can resume the match by reconnecting with the same nickname and color. If all required players reconnect within the configured timeout, the match resumes from the saved state; otherwise, the save is discarded and the server becomes available for a new game.

Implementation Choices and Design Patterns

The project follows the MVC pattern. The Model is independent from both the network layer and the view, and communicates updates through DTOs and listeners. The Controller receives client requests through common abstractions and coordinates the current phase. The View never calls the Model directly; instead, it always uses the **VirtualServer** interface, so it does not depend on the selected communication protocol.

The State pattern is used at two different levels. In the controller, the **ControllerPhase** interface represents the behavior available at a specific moment of the application flow, with implementations such as **LobbyPhase**, **InGamePhase** and **RecoveryPhase**. In the model, the game flow is also divided into phase objects, such as totem placement, offer resolution, event resolution, extra card, end round and end game phases. This keeps the rules of each step localized and makes invalid actions easier to reject depending on the current state.

Communication between Model and Controller follows an Observer-like approach through **GameListener**. When the Model produces relevant events, such as game start, totem placement, card selection, round end or game end, the controller receives a callback, saves the state when needed and notifies clients using serializable DTOs.

The leaderboard uses the Repository pattern. The **MatchResultRepository** interface separates leaderboard logic from the persistence technology. The **SqlMatchResultRepository** implementation uses JDBC and PostgreSQL, while tests can use in-memory repositories. The **RankingService** contains the application logic: recording match results, sorting by score, filtering by player count and computing the position of a specific player.

Game persistence is implemented using Gson to serialize the game state into JSON. Since the Model contains polymorphic hierarchies, such as concrete card and phase classes, the **InterfaceAdapter** stores the concrete class name and allows objects to be correctly reconstructed during deserialization. The save procedure first writes to a temporary file and then promotes it to the main save file, reducing the risk of corrupted saves in case of a crash during writing.

Communication Protocols

The project supports both Socket and RMI while preserving the same application logic through the **VirtualServer** and **VirtualView** interfaces. **VirtualServer** represents the server from the client-side view perspective, while **VirtualView** represents a connected client from the server-side controller perspective. This way, the controller and the view do not directly depend on the chosen protocol.

In the Socket protocol, the client uses `VirtualSocketServer`, which opens a TCP connection to the server on port 12345. Client-to-server requests are represented by classes extending `ClientMessage`, while server-to-client notifications extend `ServerMessage`. Messages are serialized through `ObjectOutputStream` and read through `ObjectInputStream`. On the server side, each client is handled by `SocketClientHandler`, which reads incoming messages, executes them on the `GameController` and sends responses through a dedicated outgoing queue.

In the RMI protocol, generic messages are not exchanged. The server exposes a remote object, `RMIServerAdapter`, registered in the RMI registry on port 1099 with the name `MesosServer`. The client uses `RMIClientAdapter`, which implements both `VirtualServer` and `ClientRMI`. Client actions become direct remote method calls on the server, while server updates are sent back through remote callbacks to the client. This makes RMI more declarative, whereas the Socket implementation requires an explicit message hierarchy.

Both protocols handle disconnection and recovery. If the server crashes during a match, the client attempts to reconnect within a timeout of about 20 seconds. If the server becomes available again and a saved match exists, the player can rejoin using the original nickname and color. If recovery fails or expires, the client returns to the initial screen and can start a new match.

Division of Work

The work was divided by package and functional area, while integration, debugging and final validation were carried out collaboratively. Each member was mainly responsible for a specific part of the project, but several components, especially networking, GUI and advanced features, required shared design and testing.

- **Diana Francesconi:** mainly worked on the board-related part of the Model, including board structures and related game elements. She also contributed to the Network and GUI packages and performed a broad review and update of the test suite, focusing on regression tests and consistency with the final architecture.
- **Luca Grecchi:** mainly worked on the Game and exception packages, contributing to the implementation of the core game flow, phase interaction and error handling. He also collaborated on Network and GUI development and reviewed the overall behavior of the Model to ensure consistency with the game rules.
- **Vadym Kitsul:** mainly worked on the Player and View packages, including player-related logic and user interaction through the available interfaces. He also contributed to the implementation and integration of the advanced features.
- **Andrea Markvukaj:** mainly worked on the Card and Controller packages, including card behavior, controller phases and server-side coordination of game actions. He also played a major role in the advanced features, especially by designing and implementing the database-backed leaderboard and by contributing to the correction and integration of the persistence and recovery flow.

The UML class diagram and sequence diagrams were designed collaboratively, in order to document both the static structure of the project and the main client-server interaction flows.

Testing and Coverage

The test suite includes 50 test classes and 206 methods annotated with `@Test`; one is intentionally disabled because it requires a real integration scenario with server and client processes. The regularly executed suite therefore contains 205 active JUnit tests, covering the Model, cards, board, game phases, controller phases, configuration, leaderboard, persistence utilities and selected network recovery behavior.

The tests focus on the most critical business logic: card effects, board rules, phase transitions, final scoring, controller validation, recovery behavior and leaderboard ranking. GUI interactions and complete network executions were mainly validated through manual and integration scenarios.

Coverage was measured with IntelliJ IDEA by running the full JUnit suite with Coverage:

Package	Class	Method	Line	Branch
Config	50%	100%	100%	83%
Controller	80%	55%	61%	39%
Leaderboard	100%	60%	45%	75%
Model	95%	91%	91%	81%

Coverage is strongest on the domain model, where most deterministic game rules are implemented. Lower values in other packages are mainly due to network-dependent flows, database-specific code and integration paths that are harder to exercise with isolated unit tests.